

Artificial Intelligence (AI-2002)

Further Improving Backtracking Search

Mr. Saif ur Rehman (Lecturer – AI & DS)

saifurrehman@nu.edu.pk



- What is a Constraint Satisfaction Problem (CSP)?
- What is the Constraint Propagation?
- What is the Minimum Remaining Values (MRV) heuristic?
- What is degree heuristic
- What is the least-constraining-value heuristic?

Consider a Constraint Satisfaction Problem (CSP) with three variables:

$X=\{A,B,C\}$

$D=\{1,2,3\}$

The constraints are: $A \neq B, B \neq C, A \neq C$

Find a valid assignment of values to the variables that satisfies all constraints.

Variable and value ordering: minimum-remaining-values (MRV) heuristic.

- This intuitive idea—choosing the variable with the fewest “legal” values—is called the minimum-remaining-values (MRV) heuristic.
- It also has been called the “most constrained variable” or “fail-first” heuristic.



(a)

Variable and value ordering: degree heuristic

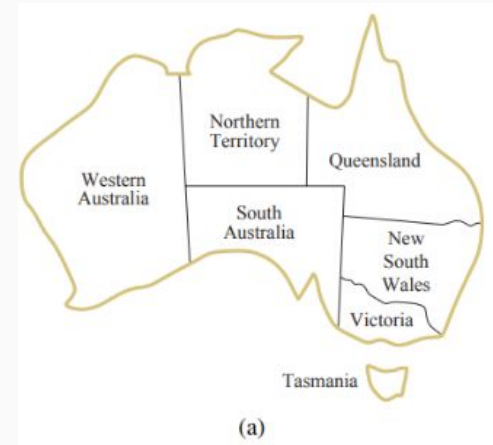
- The MRV heuristic doesn't help at all in choosing the first region to color in Australia map.
- In this case, the degree heuristic comes Degree heuristic in handy.
- Selecting the variable that is involved in the largest number of constraints on other unassigned variables.
- SA is the variable with highest degree, 5



(a)

Variable and value ordering: least-constraining-value heuristic

- Once a variable has been selected, the algorithm must decide on the order in which to examine its values.
- The least-constraining-value heuristic is effective for this.
- It prefers the value that rules out the fewest choices for the neighboring variables in the constraint graph.





Intro

- AC-3 helps reduce variable domains before starting the search.
- Inference during search can be even more powerful.
- Every time we make a choice of a value for a variable, we have a brand-new opportunity to infer new domain reductions on the neighboring variables.

Forward Checking

- One of the simplest forms of inference is called forward checking.
- Whenever a variable X is assigned, the forward-checking process establishes arc consistency for it: for each unassigned variable Y that is connected to X by a constraint, delete from Y 's domain any value that is inconsistent with the value chosen for X .

Forward Checking

	<i>WA</i>	<i>NT</i>	<i>Q</i>	<i>NSW</i>	<i>V</i>	<i>SA</i>	<i>T</i>
Initial domains							
After <i>WA</i> =red							
After <i>Q</i> =green							
After <i>V</i> =blue							

Figure 5.7 The progress of a map-coloring search with forward checking. *WA* = *red* is assigned first; then forward checking deletes *red* from the domains of the neighboring variables *NT* and *SA*. After *Q* = *green* is assigned, *green* is deleted from the domains of *NT*, *SA*, and *NSW*. After *V* = *blue* is assigned, *blue* is deleted from the domains of *NSW* and *SA*, leaving *SA* with no legal values.

Forward Checking

- Combining MRV heuristic with forward checking improves search efficiency.
- After assigning {WA=red}, MRV heuristic selects the most constrained variables (NT and SA first, then Q, NSW, and V).

Variables: A, B, C, D, E
Domains: {1,2}

Constraints:

- 1. **A ≠ B**
- 2. **B ≠ C**
- 3. **C ≠ D**
- 4. **D ≠ E**
- 5. **E ≠ A**
- 6. **B = D**

Variables: A, B, C, D, E
Domains: {1,2}

Constraints:

- 1. **A ≠ B**
- 2. **B ≠ C**
- 3. **C ≠ D**
- 4. **D ≠ E**
- 5. **E ≠ A**
- 6. **B = D**

Using Forward Checking:

Assign A = 1: FC looks at constraints involving A with unassigned variables:

- **A ≠ B:** Remove 1 from B's domain ⇒ **B becomes {2}.**
- **E ≠ A:** Remove 1 from E's domain ⇒ **E becomes {2}.**

Domains now:

- A: {1} (assigned)
- B: {2}
- C: {1,2}
- D: {1,2}
- E: {2}



Variables: A, B, C, D, E
Domains: {1,2}

Constraints:

- 1. **A ≠ B**
- 2. **B ≠ C**
- 3. **C ≠ D**
- 4. **D ≠ E**
- 5. **E ≠ A**
- 6. **B = D**

Using Forward Checking:

Assign A = 1: FC looks at constraints involving A with unassigned variables:

- **A ≠ B:** Remove 1 from B's domain ⇒ **B becomes {2}.**
- **E ≠ A:** Remove 1 from E's domain ⇒ **E becomes {2}.**

Domains now:

- A: {1} (assigned)
- B: {2}
- C: {1,2}
- D: {1,2}
- E: {2}

Using Forward Checking:

Assign B = 2 (Only possibility): FC looks at B's constraints with unassigned variables:

- **B ≠ C:** Remove 2 from C's domain ⇒ **C becomes {1}.**
- **B = D:** This forces D to equal 2 ⇒ **D becomes {2}.**

Domains now:

- A: {1} (assigned)
- B: {2} (assigned)
- C: {1}
- D: {2}
- E: {2}

Variables: A, B, C, D, E
Domains: {1,2}

Constraints:

1. **A $\neq$ B**
2. **B $\neq$ C**
3. **C $\neq$ D**
4. **D $\neq$ E**
5. **E $\neq$ A**
6. **B = D**

Using Forward Checking:

Assign A = 1: FC looks at constraints involving A with unassigned variables:

- **A $\neq$ B:** Remove 1 from B's domain $\Rightarrow$ B becomes {2}.
- **E $\neq$ A:** Remove 1 from E's domain $\Rightarrow$ E becomes {2}.

Domains now:

- A: {1} (assigned)
- B: {2}
- C: {1,2}
- D: {1,2}
- E: {2}

Using Forward Checking:

Assign B = 2 (Only possibility): FC looks at B's constraints with unassigned variables:

- **B $\neq$ C:** Remove 2 from C's domain $\Rightarrow$ C becomes {1}.
- **B = D:** This forces D to equal 2 $\Rightarrow$ D becomes {2}.

Domains now:

- A: {1} (assigned)
- B: {2} (assigned)
- C: {1}
- D: {2}
- E: {2}

Using Forward Checking:

Assign C = 1: FC looks at C's constraint with unassigned variables:

- **C $\neq$ D:** Remove 1 from D's domain. (D is {2}, so nothing changes.)

Domains now:

- A: {1} (assigned)
- B: {2} (assigned)
- C: {1} (assigned)
- D: {2}
- E: {2}

Variables: A, B, C, D, E
Domains: {1,2}

Constraints:

1. **A $\neq$ B**
2. **B $\neq$ C**
3. **C $\neq$ D**
4. **D $\neq$ E**
5. **E $\neq$ A**
6. **B = D**

Using Forward Checking:

Assign A = 1: FC looks at constraints involving A with unassigned variables:

- **A $\neq$ B:** Remove 1 from B's domain $\Rightarrow$ B becomes {2}.
- **E $\neq$ A:** Remove 1 from E's domain $\Rightarrow$ E becomes {2}.

Domains now:

- A: {1} (assigned)
- B: {2}
- C: {1,2}
- D: {1,2}
- E: {2}

Using Forward Checking:

Assign B = 2 (Only possibility): FC looks at B's constraints with unassigned variables:

- **B $\neq$ C:** Remove 2 from C's domain $\Rightarrow$ C becomes {1}.
- **B = D:** This forces D to equal 2 $\Rightarrow$ D becomes {2}.

Domains now:

- A: {1} (assigned)
- B: {2} (assigned)
- C: {1}
- D: {2}
- E: {2}

Using Forward Checking:

Assign C = 1: FC looks at C's constraint with unassigned variables:

- **C $\neq$ D:** Remove 1 from D's domain. (D is {2}, so nothing changes.)

Domains now:

- A: {1} (assigned)
- B: {2} (assigned)
- C: {1} (assigned)
- D: {2}
- E: {2}

Using Forward Checking:

Assign D = 2: FC now examines D's constraint.

- **D $\neq$ E:** Remove 2 from E's domain.
- Since E's domain was {2}, it becomes **empty**.

Domains now:

- A: {1} (assigned)
- B: {2} (assigned)
- C: {1} (assigned)
- D: {2} (assigned)
- E: {}

Forward Checking

	<i>WA</i>	<i>NT</i>	<i>Q</i>	<i>NSW</i>	<i>V</i>	<i>SA</i>	<i>T</i>
Initial domains							
After <i>WA</i> =red							
After <i>Q</i> =green							
After <i>V</i> =blue							

- Although forward checking detects many inconsistencies, it does not detect all of them. The problem is that it doesn't look ahead far enough. For example, consider the *Q*=green row of Figure. We've made *WA* and *Q* arc-consistent, but we've left both *NT* and *SA* with blue as their only possible value, which is an inconsistency, since they are neighbors.

Maintaining Arc Consistency (MAC)

- The algorithm called MAC (for Maintaining Arc Consistency) detects inconsistencies like this. After a variable X_i is assigned a value, the INFERENCE procedure calls AC-3, but instead of a queue of all arcs in the CSP, we start with only the arcs (X_j, X_i) for all X_j that are unassigned variables that are neighbors of X_i . From there, AC-3 does constraint propagation in the usual way, and if any variable has its domain reduced to the empty set, the call to AC-3 fails and we know to backtrack immediately. We can see that MAC is strictly more powerful than forward checking because forward checking does the same thing as MAC on the initial arcs in MAC's queue; but unlike MAC, forward checking does not recursively propagate constraints when changes are made to the domains of variables.

Variables: A, B, C, D, E
Domains: {1,2}

Constraints:

1. $A \neq B$
2. $B \neq C$
3. $C \neq D$
4. $D \neq E$
5. $E \neq A$
6. $B = D$

Using MAC (Maintaining Arc Consistency)

Again, assume we start by assigning $A = 1$. But with MAC, after each assignment the algorithm enforces arc consistency on **all arcs**.

1. Assign $A = 1$.

Variables: A, B, C, D, E
Domains: {1,2}

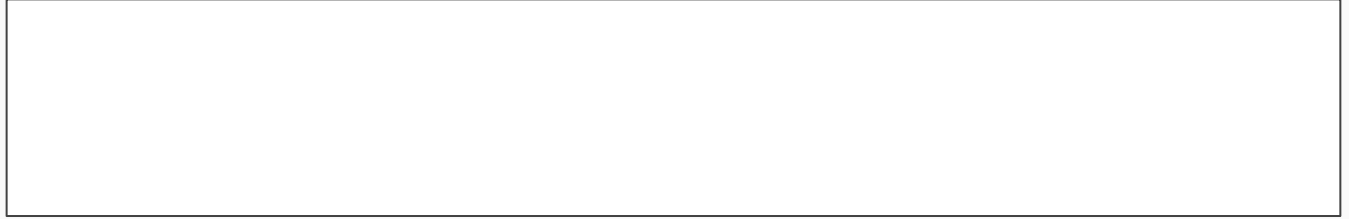
Using MAC (Maintaining Arc Consistency)

Constraints:

1. $A \neq B$
2. $B \neq C$
3. $C \neq D$
4. $D \neq E$
5. $E \neq A$
6. $B = D$

Again, assume we start by assigning $A = 1$. But with MAC, after each assignment the algorithm enforces arc consistency on **all arcs**.

1. Assign $A = 1$.
 - a. From $A \neq B$: Remove 1 from B's domain $\Rightarrow$ B becomes { 2 }.
 - b. From $E \neq A$: Remove 1 from E's domain $\Rightarrow$ E becomes { 2 }.



Variables: A, B, C, D, E
Domains: {1,2}

Using MAC (Maintaining Arc Consistency)

Constraints:

1. $A \neq B$
2. $B \neq C$
3. $C \neq D$
4. $D \neq E$
5. $E \neq A$
6. $B = D$

Again, assume we start by assigning $A = 1$. But with MAC, after each assignment the algorithm enforces arc consistency on **all arcs**.

1. Assign $A = 1$.
 - a. From $A \neq B$: Remove 1 from B's domain $\Rightarrow B$ becomes $\{2\}$.
 - b. From $E \neq A$: Remove 1 from E's domain $\Rightarrow E$ becomes $\{2\}$.
 - c. From $B = D$: Since B is now $\{2\}$, D must also be $\{2\} \Rightarrow D$ becomes $\{2\}$.



Variables: A, B, C, D, E
Domains: {1,2}

Using MAC (Maintaining Arc Consistency)

Again, assume we start by assigning **A = 1**. But with MAC, after each assignment the algorithm enforces arc consistency on **all arcs**.

Constraints:

1. **A $\neq$ B**
2. **B $\neq$ C**
3. **C $\neq$ D**
4. **D $\neq$ E**
5. **E $\neq$ A**
6. **B = D**

1. Assign A = 1.
 - a. From A $\neq$ B: Remove 1 from B's domain $\Rightarrow$ B becomes { 2 }.
 - b. From E $\neq$ A: Remove 1 from E's domain $\Rightarrow$ E becomes { 2 }.
 - c. From B = D: Since B is now { 2 }, D must also be { 2 } $\Rightarrow$ D becomes { 2 }.
 - d. From C $\neq$ D: With D now { 2 }, remove 2 from C's domain $\Rightarrow$ C becomes { 1 }.

Variables: A, B, C, D, E
Domains: {1,2}

Using MAC (Maintaining Arc Consistency)

Again, assume we start by assigning **A = 1**. But with MAC, after each assignment the algorithm enforces arc consistency on **all arcs**.

Constraints:

1. **A $\neq$ B**
2. **B $\neq$ C**
3. **C $\neq$ D**
4. **D $\neq$ E**
5. **E $\neq$ A**
6. **B = D**

1. Assign A = 1.
 - a. From A $\neq$ B: Remove 1 from B's domain $\Rightarrow$ B becomes { 2 }.
 - b. From E $\neq$ A: Remove 1 from E's domain $\Rightarrow$ E becomes { 2 }.
 - c. From B = D: Since B is now { 2 }, D must also be { 2 } $\Rightarrow$ D becomes { 2 }.
 - d. From C $\neq$ D: With D now { 2 }, remove 2 from C's domain $\Rightarrow$ C becomes { 1 }.
 - e. From D $\neq$ E: With D = { 2 } and E currently { 2 }, remove 2 from E's domain.
This leaves E with an empty domain ({ }).